

Leah Dichter

Computational Geometry: Project 2

Degenerate Inputs:

Inputs with 3+ collinear points on the convex hull caused degeneracies. Particularly, after sorting points by CCW angle to p_0 , collinear points appeared together but not necessarily in linear order. To avoid backtracking along lines and causing disconnects/issues at corner points after a line, I had to add only the first and last point on each “line segment” in the order they would appear moving CCW around the hull. Since points weren’t necessarily linearly ordered, every time I encountered three collinear points I determined which of the three points was the “middle” one by comparing their x and y coordinates, then I either removed the “middle” point from the hull (or didn’t add it), while adding/keeping the other two.

My Initializers:

```
/* ***** */
/* Initializes pts with n points on a heart. The points are in the
   range [0, WINSIZE] x [0, WINSIZE].
*/
void initialize_points_heart(vector<point2d>& pts, int n) {
    printf("\ninitialize points heart\n");
    pts.clear();
    double step = WINSIZE/n;

    point2d p;
    for (int i = 0; i < n; i++) {
        float t = step*i;
        p.x = (16*pow(sin(t), 3))*WINSIZE/50 + WINSIZE/2;
        p.y = (13*cos(t) - 5*cos(2*t) - 2*cos(3*t) - cos(4*t))*WINSIZE/50 + WINSIZE/2;
        pts.push_back(p);
    }
}
```

```
/* ***** */
/* Initializes pts with n points on 2 diagonal stripes. The points are in the
   range [0, WINSIZE] x [0, WINSIZE].
*/
void initialize_points_stripes(vector<point2d>& pts, int n) {
    printf("\ninitialize points stripes\n");
    pts.clear();
    double step = WINSIZE/n;
    n = n/2;

    point2d p;
    point2d q;
    for (int i = 0; i < n; i++) {
        float t = step*i;
        p.x = t + WINSIZE/4;
        p.y = t + WINSIZE/5;
        q.x = t + WINSIZE/4;
        q.y = t + 2*WINSIZE/5;
        pts.push_back(p);
        pts.push_back(q);
    }
}
```

Other Initializers:

```
/* ***** */
/* Initializes pts with n points on the sides of a square, rotated 45 degrees from the x and y axes. The square
   The square is centered in the window with sidelength WINSIZE/4 * SQRT(2) so that it has width WINSIZE/2
*/
void initialize_points_diamond(vector<point2d>& pts, int n) {
    printf("\ninitialize points square\n");

    //clear the vector just to be safe
    pts.clear();
    int width = WINSIZE / 2; //from left corner to right corner (so sidelength is sqrt(2)/2 times this)
    int center = WINSIZE / 2;

    //4 sides of the square w/ evenly distributed points, but the square is rotated 45 degrees CCW
    //for loop evenly distributes points along each of the four edges at once, distributing in clockwise direction
    for (int i = 0; i < n/4; i++){
        double offset = (double)2 / n * width * i; // to distribute 1/4 of all the points at equal intervals

        point2d pNW; //point on northwest edge
        pNW.x = center - ((double)width / 2) + offset;
        pNW.y = center + offset;
        pts.push_back(pNW);

        point2d pSE; //point on southeast edge
        pSE.x = center + ((double)width / 2) - offset;
        pSE.y = center - offset;
        pts.push_back(pSE);

        point2d pSW; //point on southwest edge
        pSW.x = center - offset;
        pSW.y = center - ((double)width / 2) + offset;
        pts.push_back(pSW);

        point2d pNE; //point on northeast edge
        pNE.x = center + offset;
        pNE.y = center + ((double)width / 2) - offset;
        pts.push_back(pNE);
    }

    //put the extra points in the middle:
    for (int i = 0; i < (n % 4); i++){
        point2d p;
        p.x = center - ((double)width / 2) + (i+1)*(width / 4);
        p.y = center;
        pts.push_back(p);
    }
}
```

```
/*this initializer will always make 25 hardcoded points that are meant to test collinearity
 * user can input a number for parameter n and n points will be created in addition to the
 * 25 original points
 */
void initializer_square(vector<point2d>& pts, int n_additional_points){

    pts.clear(); //should be empty, but clear it to be safe
    for(int i = 200; i <= 300; i+=20){
        for(int j = 200; j <= 300; j+=20){
            point2d p;
            p.x = i;
            p.y = j;
            pts.push_back(p);
        }
    }

    //we are using user input to create n points in addition to our hard coded collinear points
    for (int i=0; i<(n_additional_points); i++) {
        point2d p_rand;
        p_rand.x = (int)200 + random() % ((int)(100));
        p_rand.y = (int)200 + random() % ((int)(100));
        pts.push_back(p_rand);
    }
}
```

```

// Function to initialize points in a hexagon
void initialize_points_hexagon(vector<point2d>& pts, int n) {
    pts.clear();
    // Define the center of the hexagon
    double centerX = WINDOWSIZE / 2.0;
    double centerY = WINDOWSIZE / 2.0;
    // Set the radius of the hexagon (distance from center to any vertex)
    double radius = WINDOWSIZE / 4.0;
    // Number of sides for the hexagon
    int numSides = 6;
    // Angle between each vertex
    double angleStep = 2 * M_PI / numSides;
    // Add hexagon vertices
    vector<point2d> hexagonVertices;
    for (int i = 0; i < numSides; ++i) {
        point2d p;
        p.x = centerX + radius * cos(i * angleStep);
        p.y = centerY + radius * sin(i * angleStep);
        pts.push_back(p); // Store the hexagon vertices in the pts list
        hexagonVertices.push_back(p); // Store the hexagon vertices separately for point-in-polygon check
    }
    // Add random points inside the hexagon boundary
    int generatedPoints = 0;
    while (generatedPoints < n) {
        point2d p;
        // Generate a random point within a bounding box around the hexagon
        p.x = centerX + (((double)rand() / RAND_MAX) * 2 - 1) * radius;
        p.y = centerY + (((double)rand() / RAND_MAX) * 2 - 1) * radius;
        if (is_point_on_polygon(p, hexagonVertices)) {
            pts.push_back(p);
            generatedPoints++;
        }
    }
    printf("Hexagon: initialized with %lu points\n", pts.size());
}

```

```

// Function to initialize points in a wave shape based on the sine function
void initialize_points_wave(vector<point2d>& pts, int n){
printf("\ninitialize points wave\n");
    //clear the vector just to be safe
    pts.clear();
    double step = (double)WINDOWSIZE / n;
    double amplitude = 100; // Height of the wave
    double frequency = 0.1; // Controls the number of waves
    point2d p;
    for (int i = 0; i < n; ++i) {
        p.x = i * step;
        p.y = WINDOWSIZE / 2 + amplitude * sin(frequency * p.x);
        pts.push_back(p);
    }
}

```

```

void initialize_points_honeycomb(vector<point2d> &pts, int n)
{
    printf("\ninitialize points honeycomb\n");
    // clear the vector just to be safe
    pts.clear();

    if (n < 6)
    {
        printf("n must be at least 6\n");
        return;
    }

    point2d p;
    int step = WINDOWSIZE / 3;

    // center of the honeycomb
    double centerX = WINDOWSIZE / 2;
    double centerY = WINDOWSIZE / 2;

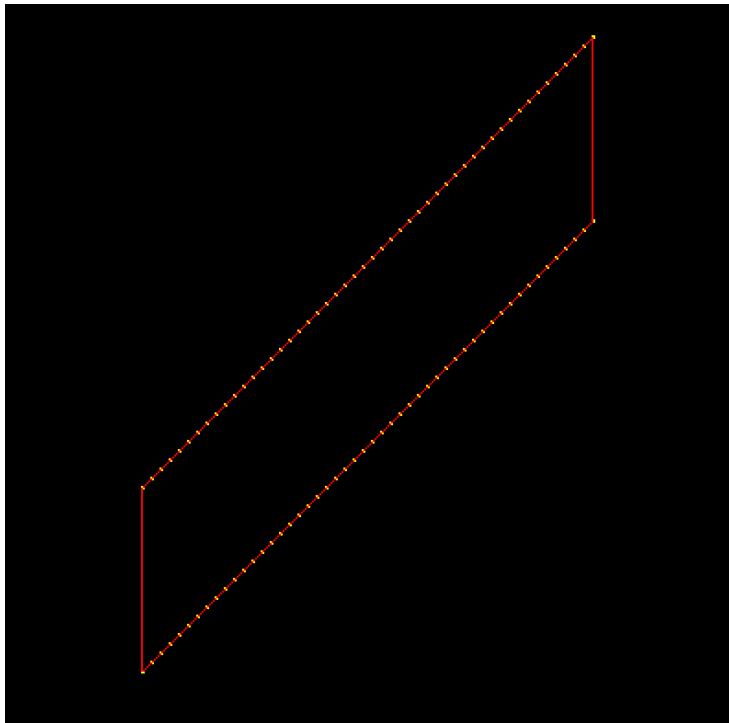
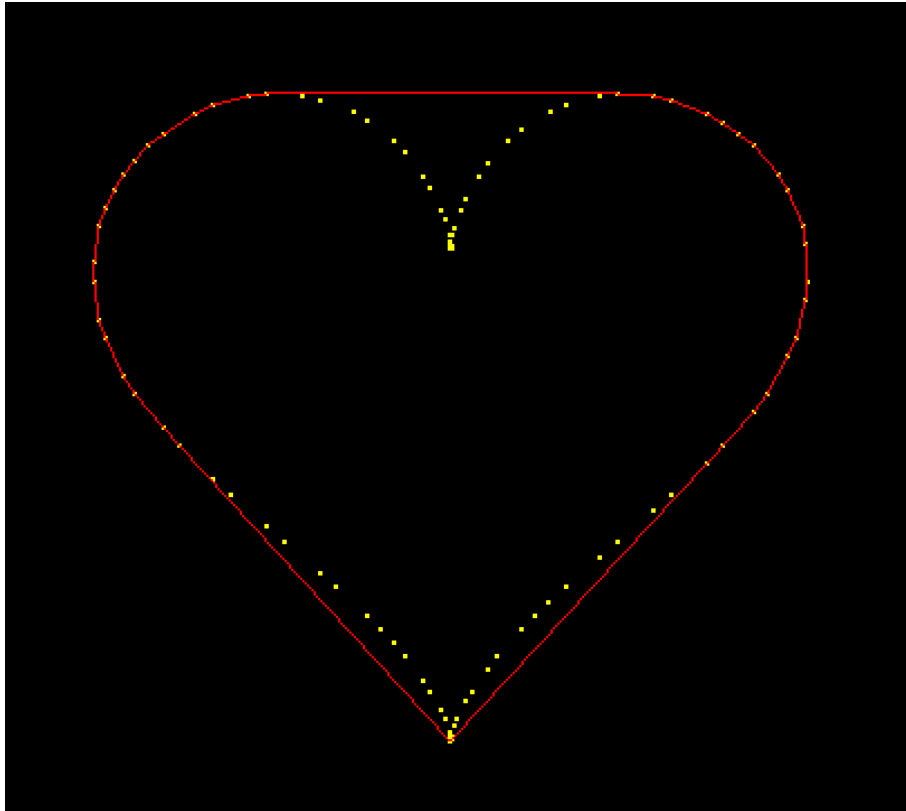
    // generate points for the hexagon
    for (int i = 0; i < n; i++)
    {
        double angle = 2 * M_PI * i / n;

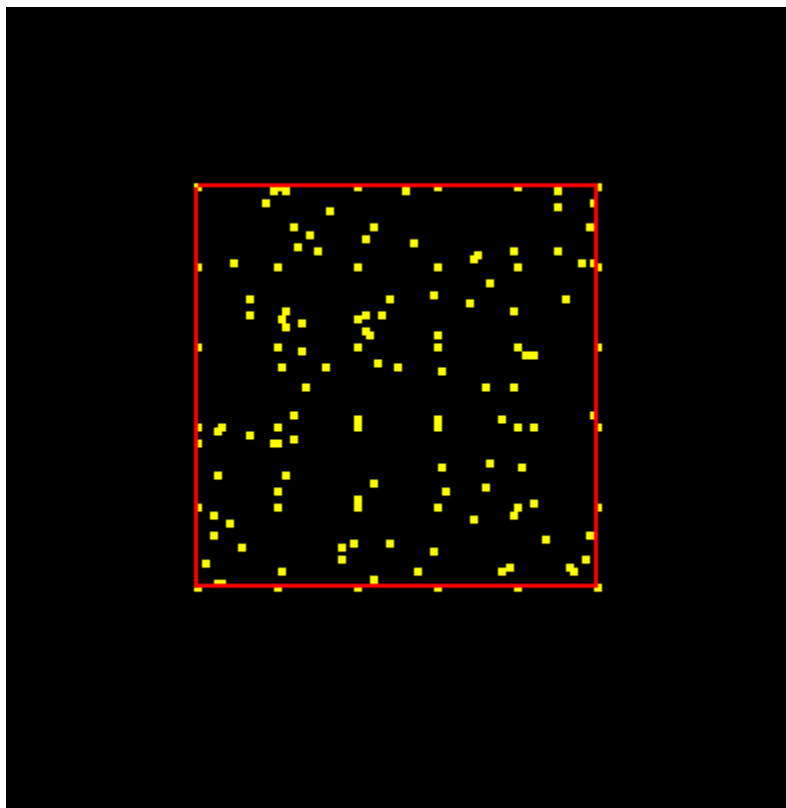
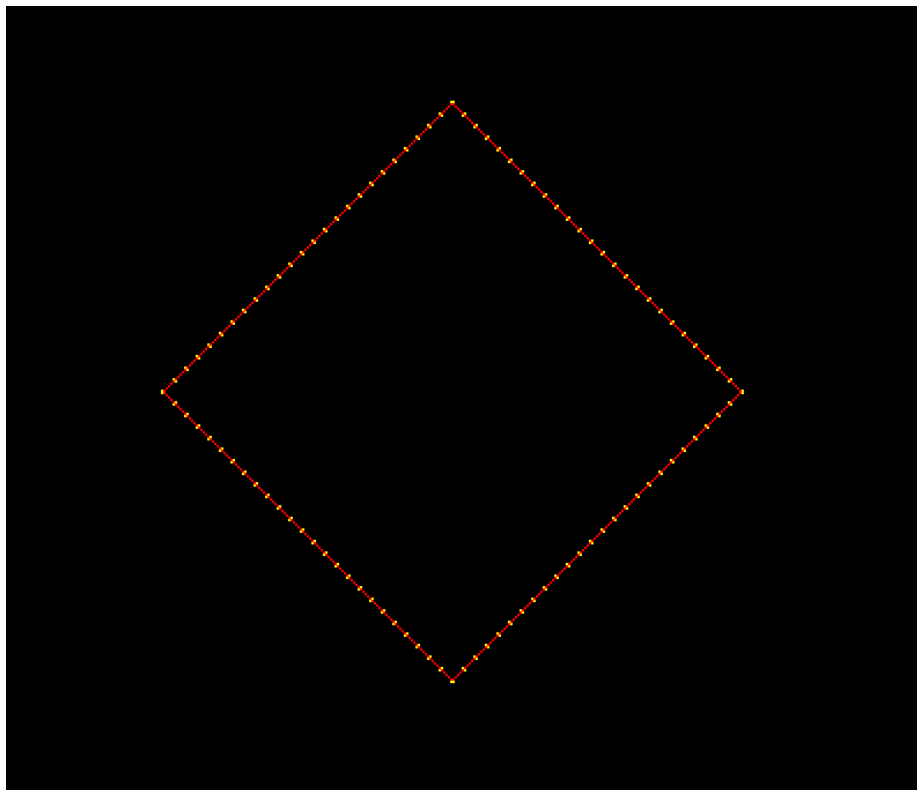
        if (i % (n / 6) == 0)
        {
            // Corner points
            p.x = centerX + step * cos(angle);
            p.y = centerY + step * sin(angle);
        }
        else
        {
            // points along the edges
            double t = (i % (n / 6)) / (double)(n / 6);
            int corner = i / (n / 6);
            double x1 = centerX + step * cos(2 * M_PI * corner / 6);
            double y1 = centerY + step * sin(2 * M_PI * corner / 6);
            double x2 = centerX + step * cos(2 * M_PI * (corner + 1) / 6);
            double y2 = centerY + step * sin(2 * M_PI * (corner + 1) / 6);
            p.x = x1 + t * (x2 - x1);
            p.y = y1 + t * (y2 - y1);
        }

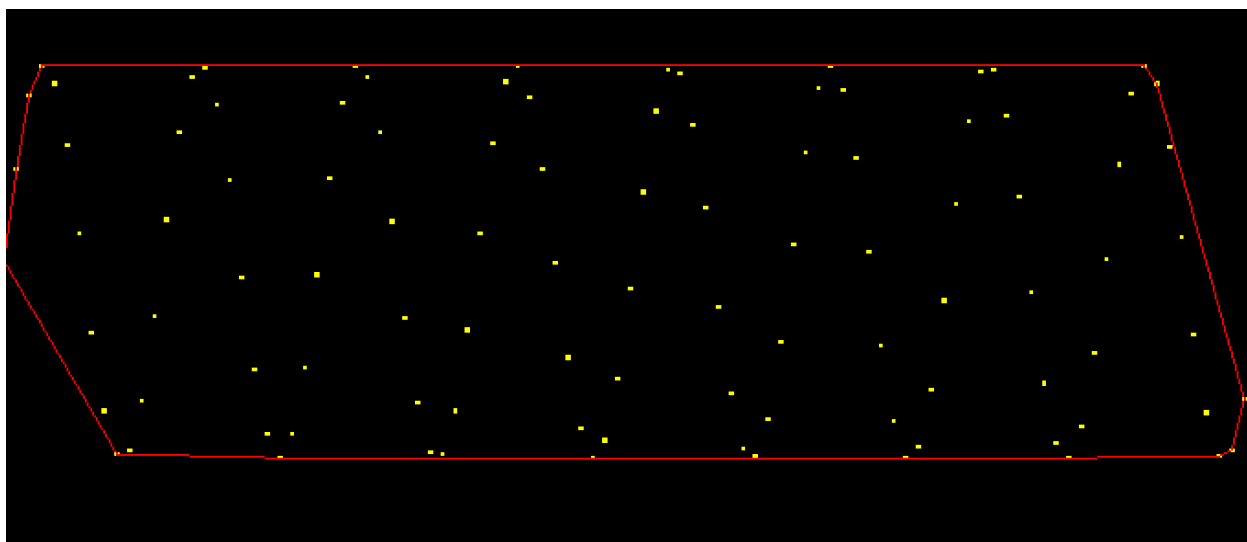
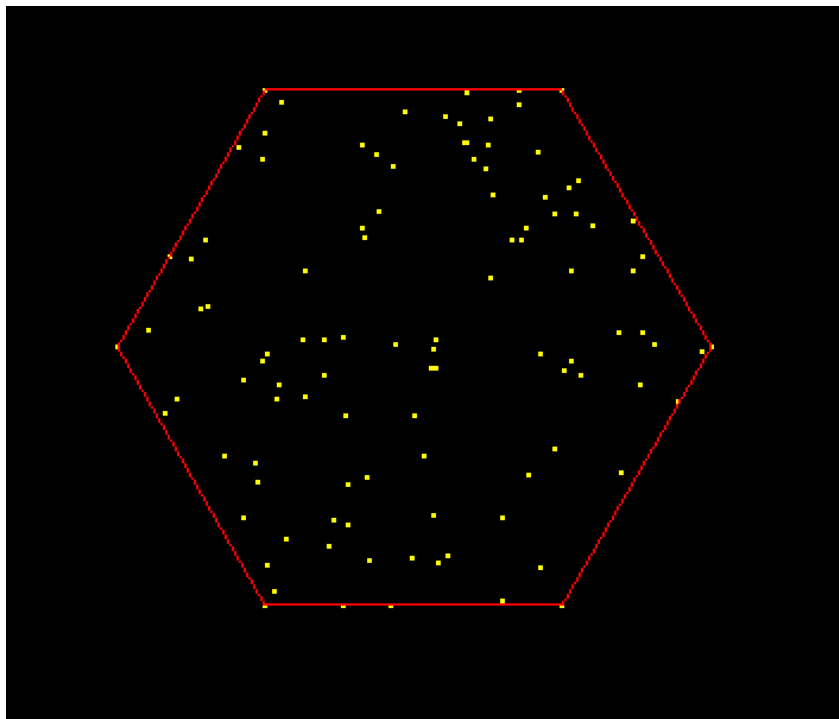
        pts.push_back(p);
    }
}

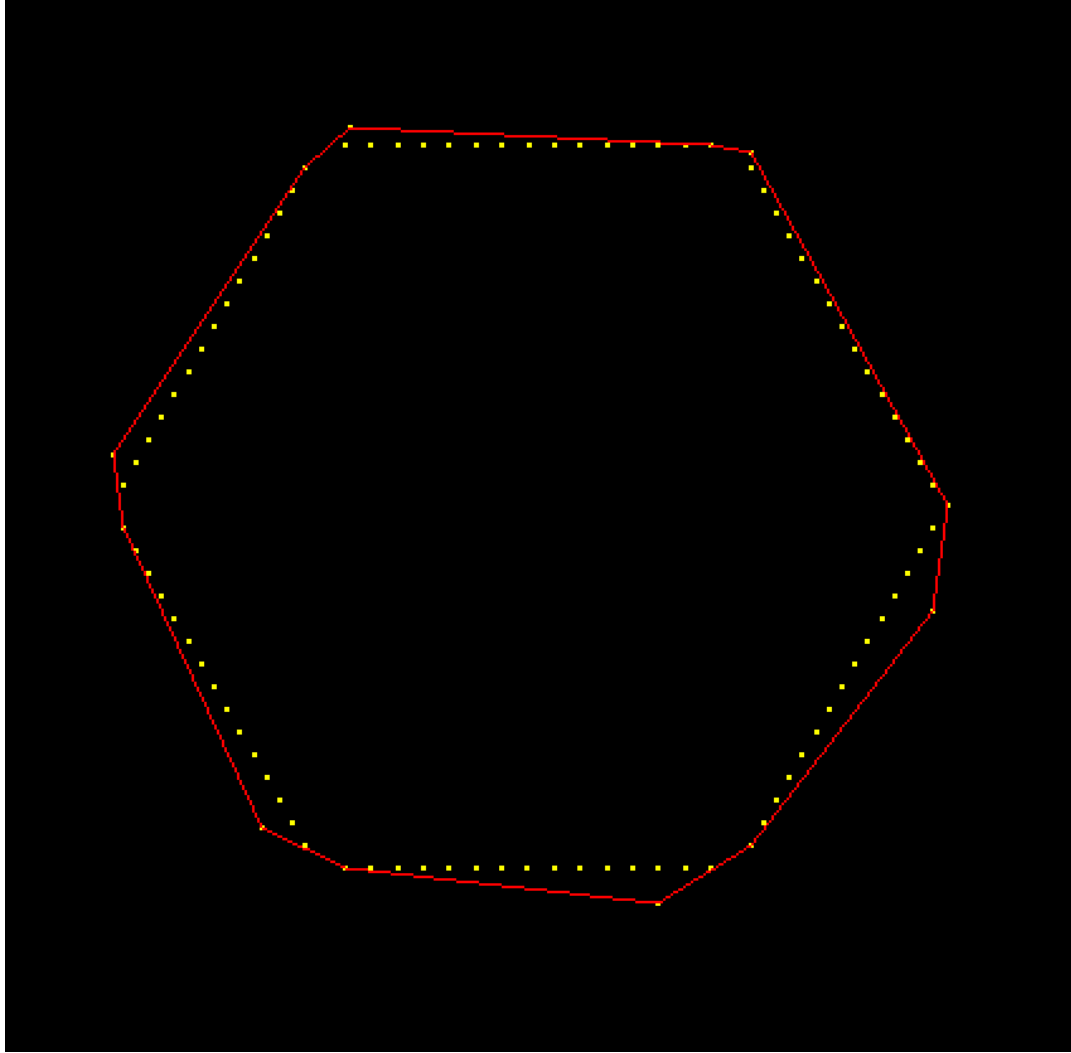
```

Convex Hull (For each initializer, in order):







**Reflections:**

I thought that the actual coding in this project wasn't particularly hard, but it was challenging in the sense that you really had to have a good visual understanding of what your algorithm was doing at each step in order to handle degenerate cases (and understand why certain cases were causing degeneracies). I wish I'd thought more ahead of time about possible degenerate cases, because I instead addressed them as they arose, and I think I could have saved myself time and developed a stronger initial intuition for the algorithm had I tried harder to predict degenerate cases up front. I probably spent around 3 hours coding and testing, and "thinking" was interspersed throughout the week.